

Projet Informatique de 2ème année

Groupe 32

EX-Libris

Étudiants :

Arnaud BICHON
Axel LECLERCQ
Cécile LOCATELLI
Moussa SOULAMA
Anne-Camille TAMPÉ
Paul FOURCADE

Encadrant :

Elwenn JOUBREL

Table des matières

1	Introduction	3
2	Cahier des Charges	5
2.1	Diagramme de Gantt	5
2.2	Diagramme de cas d'utilisation	6
3	Maquette de l'application	7
3.1	Diagrammes de séquence	7
3.2	Diagramme de classes	10
3.2.1	Classes Business Object	10

1 Introduction

Depuis plusieurs années, le numérique a profondément transformé la manière dont nous consommons et partageons nos loisirs culturels. Le cinéma dispose de Letterboxd, la musique de Last.fm : des plateformes communautaires qui permettent à chacun de garder une trace de ses découvertes, de les noter, de les commenter et d'échanger avec d'autres passionnés. La lecture, pourtant, reste étonnamment mal servie sur ce plan. Le principal service existant, Goodreads, appartient à Amazon, ce qui pose des questions en matière d'indépendance éditoriale et d'exploitation des données personnelles des lecteurs. Il n'existe pas aujourd'hui d'alternative crédible, ouverte et centrée sur l'expérience du lecteur plutôt que sur la vente de livres.

C'est de ce constat qu'est né le projet Ex-Libris : une application de suivi de lecture pensée comme un véritable réseau social pour les lecteurs. L'objectif n'est pas de reconstruire un catalogue de livres, un travail déjà accompli et maintenu par la plateforme collaborative Open Library, qui référence plusieurs millions d'ouvrages en accès libre. Ce que Open Library ne propose pas, en revanche, c'est justement la dimension humaine de la lecture, qui a lu quel livre, quand, avec quel avis, et quelles recommandations peuvent en être tirées. C'est précisément cette couche que notre application vient ajouter.

Concrètement, Ex-Libris doit permettre à un utilisateur de créer un compte, de rechercher un livre et d'en consulter la fiche détaillée (couverture, auteur, année de parution, éditions disponibles...), de l'ajouter à sa bibliothèque personnelle avec un statut de lecture (à lire, en cours, lu, abandonné), de le noter et d'en rédiger une critique. Au-delà de cette gestion individuelle, l'application intègre une dimension sociale : chaque utilisateur peut consulter le profil d'un autre lecteur, s'abonner à son activité et découvrir sa bibliothèque, ses critiques et ses coups de cœur. Une fonctionnalité de recommandation, construite à partir des lectures, des notes et des affinités entre utilisateurs, viendra enrichir cette expérience en aidant chacun à découvrir nos prochaines lectures.

D'un point de vue technique, le projet repose sur le développement d'une API REST construite avec FastAPI, exploitant les données de l'API publique Open Library et s'appuyant sur une base de données PostgreSQL pour la persistance des informations propres à notre application (comptes, bibliothèques, critiques, relations entre utilisateurs). Le langage retenu pour l'ensemble du développement est Python, dans une logique de programmation orientée objet, ce qui permet une architecture claire, modulaire et évolutive. Un client pourra, dans un second temps, venir compléter cette API pour offrir une véritable expérience utilisateur.

La question à laquelle ce projet cherche à répondre peut ainsi se formuler de la manière suivante : comment concevoir une architecture logicielle capable de restituer, de façon fiable et évolutive, la dimension à la fois personnelle et sociale de la lecture, à partir d'un catalogue externe que l'on ne maîtrise pas ?

Ce dossier présente dans un premier temps le cahier des charges du projet, les choix et arbitrages qui ont guidé nos réflexions, ainsi que l'organisation retenue au sein de notre équipe.

Il détaille ensuite le modèle de conception élaboré pour répondre à ces besoins, à travers les différents diagrammes UML réalisés.

2 Cahier des Charges

2.1 Diagramme de Gantt

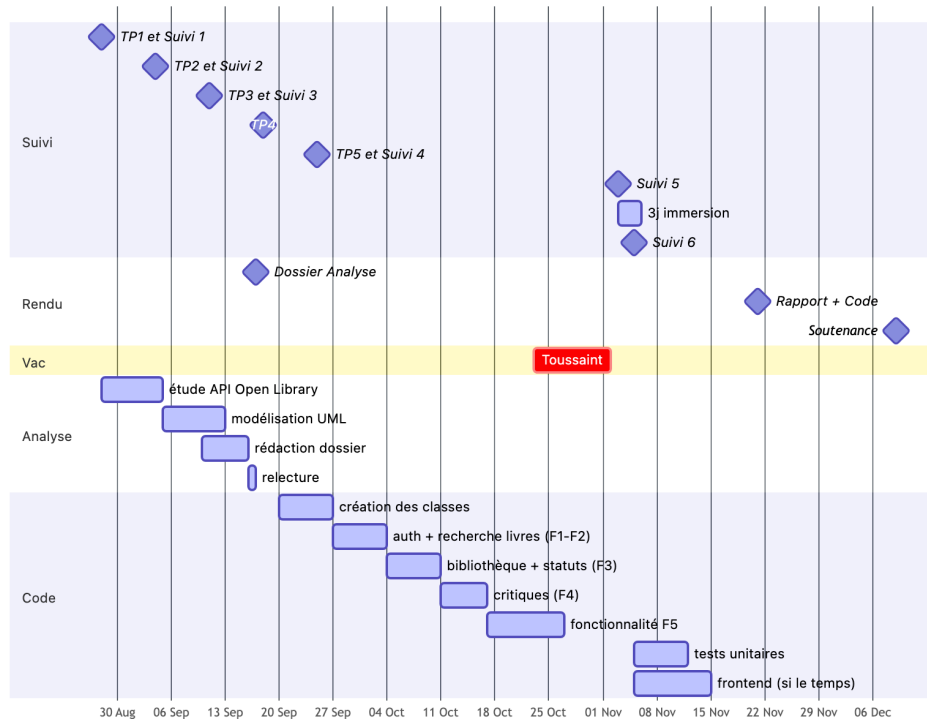


Figure 11 – Diagramme de Gantt

Notre planning se découpe en deux grandes phases. D'abord la phase d'analyse, de fin août à fin septembre : nous commençons par étudier l'API Open Library, puis nous réalisons l'ensemble de la modélisation UML afin de bien comprendre comment notre application va s'articuler et comment la construire.

Une fois cette base posée, nous passons à la phase de code, d'octobre à début novembre. Nous avons choisi de développer les fonctionnalités par ordre de difficulté croissante : d'abord les classes de base, puis l'authentification et la recherche de livres (F1-F2), la bibliothèque avec les statuts de lecture (F3), les critiques (F4), et enfin notre fonctionnalité F5 (les recommandations), qui nécessitera davantage de temps du fait de sa complexité. Les tests unitaires et le frontend interviendront en dernier, le frontend n'étant pas prioritaire, sera développé uniquement s'il nous reste du temps.

2.2 Diagramme de cas d'utilisation

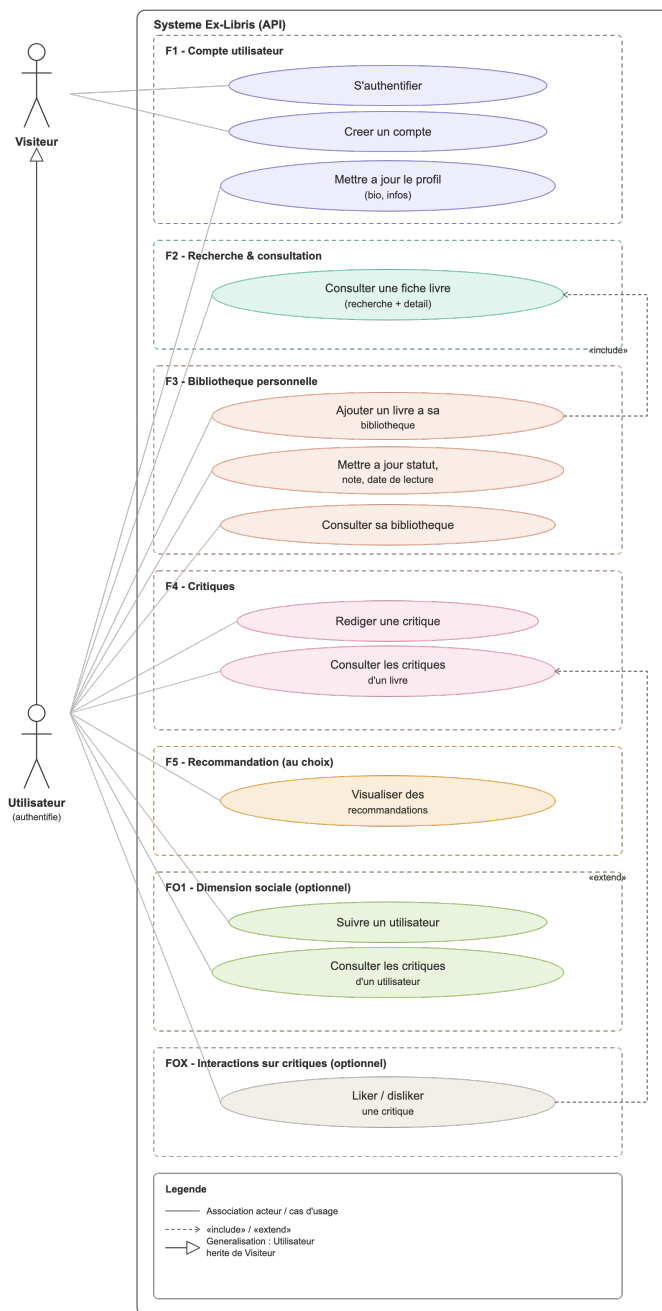


Figure 11 – Diagramme de cas d'utilisation

3 Maquette de l'application

3.1 Diagrammes de séquence

Les diagrammes de séquence suivant permettent de détailler la chronologie des échanges entre les différents composants de notre architecture (Utilisateur, Front End, Back End, DAO, API externe) pour trois fonctionnalités de l'application : la suppression d'une fiche de lecture, la sélection d'un livre et la demande de recommandations personnalisées.

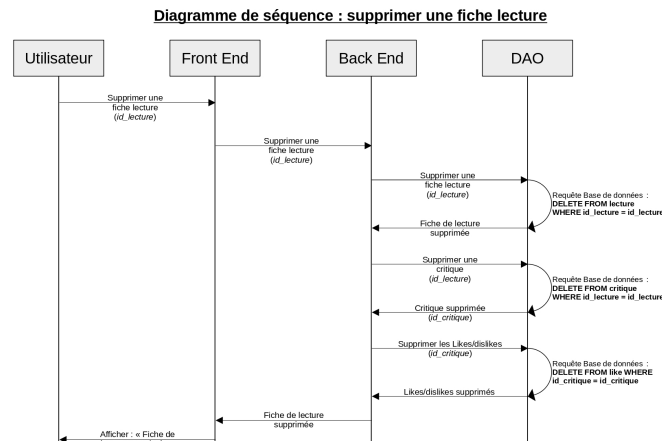


Figure 11 – Diagramme de séquence : supprimer une fiche de lecture

Ce diagramme illustre le processus de suppression en cascade déclenché lorsqu'un utilisateur souhaite retirer un livre de sa bibliothèque. La demande transite du Front End vers le Back End, qui orchestre alors une série de suppressions au niveau de la DAO : la fiche de lecture elle-même est d'abord supprimée (`DELETE FROM lecture WHERE id_lecture = ...`), puis la critique associée à cette lecture (`DELETE FROM critique WHERE id_lecture = ...`), et enfin les likes et dislikes rattachés à cette critique (`DELETE FROM like WHERE id_critique = ...`). Cet enchaînement met en évidence les dépendances entre nos tables : une lecture peut porter une critique, elle-même pouvant porter des réactions. La suppression doit donc respecter cet ordre pour éviter toute violation de contrainte d'intégrité référentielle. Une fois la cascade terminée, une confirmation remonte jusqu'à l'utilisateur sous la forme d'un message « Fiche de lecture supprimée ».

Diagramme de séquence : sélectionner un livre

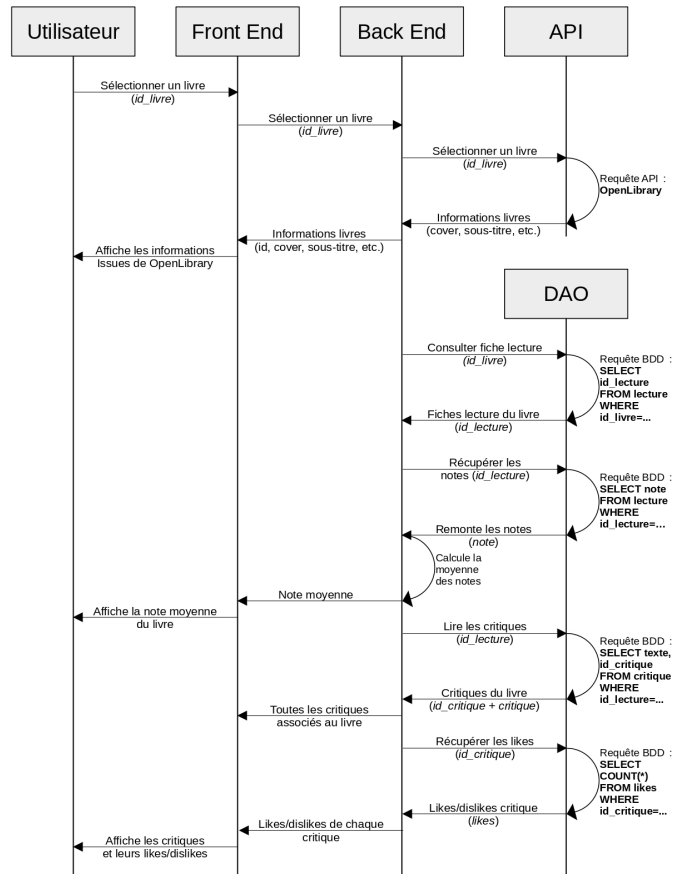


Figure 11 – Diagramme de séquence : sélectionner un livre

Ce diagramme décrit l’affichage de la fiche détaillée d’un livre, qui combine des données externes et des données propres à notre application. Lorsque l’utilisateur sélectionne un livre, le Back End interroge dans un premier temps l’API Open Library afin de récupérer les informations bibliographiques (couverture, sous-titre, etc.) qui ne sont pas stockées localement. En parallèle, le Back End consulte la DAO pour extraire les données internes à Ex-Libris associées à ce livre : les fiches de lecture existantes, les critiques rédigées par les utilisateurs, ainsi que les likes et dislikes rattachés à chacune d’elles. Une agrégation est également effectuée côté Back End : les notes attribuées par les lecteurs sont récupérées puis moyennées pour produire une note globale du livre. L’ensemble de ces informations (Open Library, critiques, likes/dislikes, note moyenne) est ensuite consolidé et renvoyé au Front End pour affichage. Ce diagramme illustre bien la logique de notre architecture : Open Library reste la source de vérité pour le contenu bibliographique, tandis que notre base de données ne conserve que ce qui relève de l’activité des utilisateurs.

Ce diagramme correspond à notre fonctionnalité F5, un système de recommandation basé sur les affinités entre lecteurs. Le processus se déroule en plusieurs temps. Le Back End récupère d'abord, via la DAO, la liste des livres déjà lus par l'utilisateur demandeur. Pour chacun de ces livres, une requête identifie les autres utilisateurs ayant également lu ce même livre. Le Back End calcule ensuite, lecteur par lecteur, le nombre de livres en commun avec l'utilisateur initial, ce qui permet de dégager un sous-ensemble de « lecteurs similaires » selon un critère de recoupement. Pour ces lecteurs sélectionnés, l'application récupère l'ensemble de leurs livres lus ainsi que les notes associées. Le Back End retire alors de cette liste les livres déjà lus par l'utilisateur initial et ne conserve que les livres les mieux notés ou, en l'absence de signal suffisant, propose des livres bien notés au hasard parmi ceux restants. Une fois la sélection finale établie, les informations bibliographiques des livres recommandés sont récupérées auprès de l'API Open Library avant d'être affichées à l'utilisateur.

3.2 Diagramme de classes

3.2.1 Classes Business Object

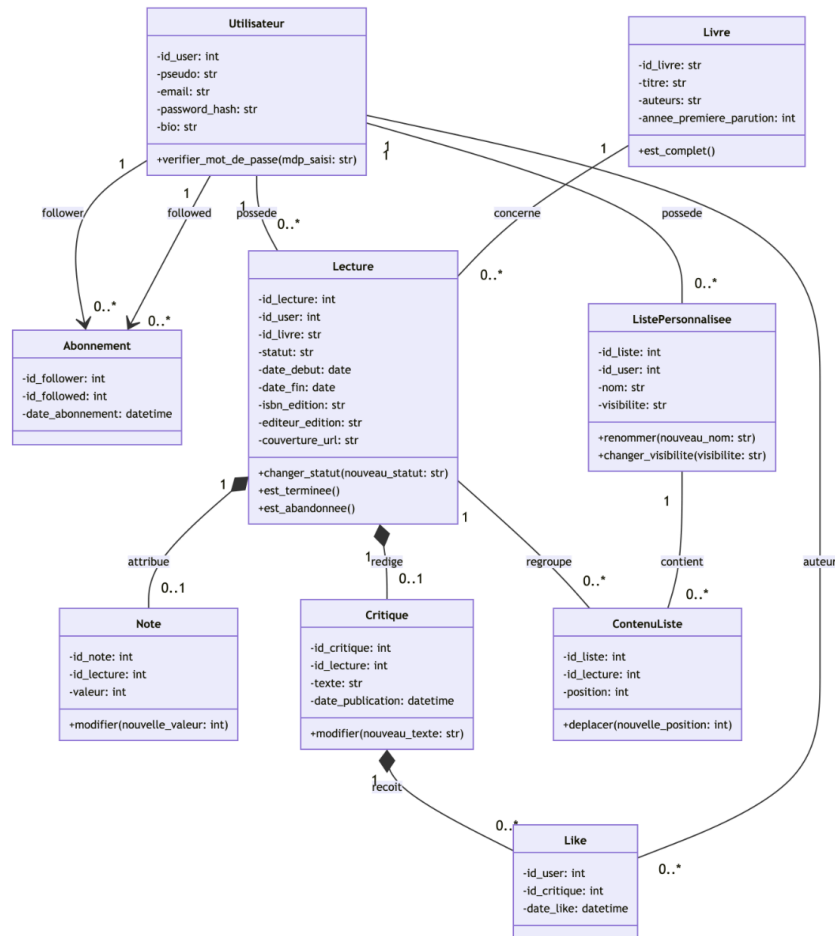


Figure 11 – Diagramme de classe : Business Object

partie 1 : Cahier des Charges partie 2 : Maquette de l'application

Suivi projet Ex-Libris

Fait

- Arno : diagramme de séquence (partie 2)
- Paul : diagramme de Gantt (partie 1) paul le caca qui pue berkklkkk

- Cecile : Diagramme de cas d'utilisation (partie 1)

En cours

- Moussa : diagramme BDD (partie 2)
- Axel : diagramme d'activité (partie 1)
- Anne-Camille : diagramme de classe business (détail + explication des classes) (partie 2)
- Paul : Introduction

À faire

- Partie répartition des tâches et présentation de l'équipe (partie 2)
- Diagramme de package (DAO / Services / Business / View) (partie 2)
- Détail d'un service (partie 2)
- Présentation de l'API (partie 1) ...